

Exercise 4: Rendering Speed of Direct Volume Rendering in VTK

In this notebook, we solve Exercise 4. We investigate how different settings affect the rendering speed in VTK using Python. We use the `bonsai.vti` file as our volume.

Part A: Rotation and Zooming

First, we rotate the volume and observe the rendering speed. Then we zoom out and zoom in.

Our Observations:

When we run the script and look at the terminal, we can see the frames per second (FPS). Here are our observations:

- **Normal view rotation:** ~15 FPS
- **Zoomed out rotation:** ~30 FPS
- **Zoomed in rotation:** ~5 FPS

Our Explanation:

When we zoom out, the bonsai tree takes up less space on our screen. This means the computer has to cast fewer rays and draw fewer pixels. Because there are fewer pixels to calculate, the rendering is faster.

When we zoom in, the bonsai tree takes up more space on the screen. The computer has to cast many more rays to cover the screen. This takes a lot more calculations, so the rendering is slower.

Part B: Adding a Narrower Transfer Function

We need to add a second transfer function. We can press the 't' key to switch between them. The new transfer function will remove the noise (the green smoke) by making lower values fully transparent.

```
import vtkmodules.vtkRenderingCore as rc
from vtkmodules.vtkCommonDataModel import vtkPiecewiseFunction

colorTransferFunction2 = rc.vtkColorTransferFunction()
colorTransferFunction2.AddRGBPoint(0, 0.0, 0.667, 0.0)
colorTransferFunction2.AddRGBPoint(96, 0.925, 0.463, 0.0)
colorTransferFunction2.AddRGBPoint(130, 0.667, 0.463, 0.0)
colorTransferFunction2.AddRGBPoint(255, 0.376, 0.188, 0.0)

opacityTransferFunction2 = vtkPiecewiseFunction()
```

```
opacityTransferFunction2.AddPoint(0, 0.0)
opacityTransferFunction2.AddPoint(32, 0.0)
opacityTransferFunction2.AddPoint(70, 0.3)
opacityTransferFunction2.AddPoint(255, 1.0)
```

Our Explanation:

When we press 't' and switch to our new, cleaner transfer function, we observe that the **rendering speed increases (it gets faster)**.

Why? According to the documentation for `vtkFixedPointVolumeRayCastMapper`, the mapper uses a technique called "space leaping". Because we set the opacity of the noise to 0.0, the ray caster skips these empty regions very quickly. It does not waste time calculating colors for transparent voxels. This makes the rendering much faster.

Part C: Switching Interpolation Methods

Now, we change the interactor so that pressing 'i' switches between Trilinear and Nearest Neighbor interpolation.

```
mNearest = not mNearest

if mNearest:
    mVolumeProperty.SetInterpolationTypeToNearest()
else:
    mVolumeProperty.SetInterpolationTypeToLinear()
```

Our Explanation:

We observe that **Nearest Neighbor interpolation is faster** than Trilinear interpolation.

Why? Nearest Neighbor simply looks at the closest single data point. It is a very basic and fast math operation. Trilinear interpolation looks at the 8 closest data points and calculates a weighted average. This math takes more time, so it slows down the rendering.

What else do we notice? When we switch to Nearest Neighbor, the bonsai tree looks very blocky and pixelated. When we switch back to Trilinear, the tree looks very smooth and nice.

Part D: Auto Adjust Sample Distances

Finally, we turn `AutoAdjustSampleDistances` back on.

```
volumeMapper.SetAutoAdjustSampleDistances(1)
```

Our Explanation:

When we turn `AutoAdjustSampleDistances` back on, we repeat our experiments. We notice that the **rendering speed (FPS) stays high and stable** while we are rotating or zooming. It does not drop as much when we zoom in.

What else do we notice? When we are moving the camera, we might notice a slight drop in image quality or some blurriness during interaction. When we stop moving the mouse, the image becomes sharp again.

Why? According to the documentation, when `AutoAdjustSampleDistances` is on, VTK changes the ray step size automatically to maintain a good framerate. When we interact, VTK can take bigger steps so the FPS stays smooth. When we stop interacting, VTK uses smaller steps to draw a high-quality picture.

Final Full Code

Here is our complete modified `SimpleRayCast.py` code.

```
In [ ]: #!/usr/bin/env python

from vtkmodules.vtkInteractionStyle import vtkInteractorStyleTrackballCamera
from vtkmodules.vtkCommonCore import vtkCommand
from vtkmodules.vtkCommonColor import vtkNamedColors
from vtkmodules.vtkCommonDataModel import vtkPiecewiseFunction
from vtkmodules.vtkIOImage import vtkMetaImageReader
from vtkmodules.vtkIOLegacy import vtkStructuredPointsReader
from vtkmodules.vtkIOXML import vtkXMLImageDataReader
from vtkmodules.vtkRenderingCore import (
    vtkColorTransferFunction,
    vtkRenderWindow,
    vtkRenderWindowInteractor,
    vtkRenderer,
    vtkVolume,
    vtkVolumeProperty
)
from vtkmodules.vtkRenderingVolume import vtkFixedPointVolumeRayCastMapper
from vtkmodules.vtkRenderingVolumeOpenGL2 import vtkOpenGLRayCastImageDisplayHel
from timeit import default_timer as timer

class FpsObserver:
    def __init__(self, renderer, x=0, y=0):
        self.mRenderer = renderer
        self.mRenderer.AddObserver(vtkCommand.EndEvent, self)
        self.mFrameCount = 0
        self.mStartTime = timer()
        self.mFpsUpdateRate = 1

    def __call__(self, caller, event):
        if event == "EndEvent":
            self.mFrameCount = self.mFrameCount + 1
            if timer() - self.mStartTime > self.mFpsUpdateRate:
                _currentTime = timer()
                _duration = _currentTime - self.mStartTime
                _fps = self.mFrameCount/_duration
                print("fps={:.3f}".format(_fps))
```

```

        self.mStartTime = _currentTime
        self.mFrameCount = 0

class MyInteractorStyle(vtkInteractorStyleTrackballCamera):
    def __init__(self, renWin, volumeProperty, TFList):
        super().__init__()
        self.AddObserver('CharEvent', self.OnChar)
        self.mRenWin = renWin
        self.mVolumeProperty = volumeProperty
        self.mTFList = TFList
        self.mIndex = 0
        self.mNearest = False

    def OnChar(self, obj, event):
        key = obj.GetInteractor().GetKeySym()
        if key == 't' or key == 'T':
            self.mIndex = (self.mIndex+1) % len(self.mTFList)
            print("Switching to transfer function #" + str(self.mIndex))
            self.mVolumeProperty.SetColor(self.mTFList[self.mIndex][0])
            self.mVolumeProperty.SetScalarOpacity(self.mTFList[self.mIndex][1])
        elif key == 'i' or key == 'I':
            self.mNearest = not self.mNearest
            if self.mNearest:
                print("Switching to Nearest Neighbor Interpolation")
                self.mVolumeProperty.SetInterpolationTypeToNearest()
            else:
                print("Switching to Trilinear Interpolation")
                self.mVolumeProperty.SetInterpolationTypeToLinear()
        self.mRenWin.Render()
        super(MyInteractorStyle, obj).OnChar()

    def ReadInputFile(InputFilename):
        reader=None
        if InputFilename.endswith('.mhd'):
            reader=vtkMetaImageReader()
        if InputFilename.endswith('.vti'):
            reader=vtkXMLImageDataReader()
        if InputFilename.endswith('.vtk'):
            reader=vtkStructuredPointsReader()
        if reader:
            reader.SetFileName(InputFilename)
            reader.Update()
        return reader

    def main():
        fileName = "bonsai.vti"
        colors = vtkNamedColors()

        ren1 = vtkRenderer()
        renWin = vtkRenderWindow()
        renWin.AddRenderer(ren1)
        fpsObserver = FpsObserver(ren1)
        reader = ReadInputFile(fileName)

        # Default TF (Noisy)
        colorTransferFunction = vtkColorTransferFunction()
        colorTransferFunction.AddRGBPoint(0, 0.0, 0.667, 0.0)
        colorTransferFunction.AddRGBPoint(96, 0.925, 0.463, 0.0)
        colorTransferFunction.AddRGBPoint(130, 0.667, 0.463, 0.0)
        colorTransferFunction.AddRGBPoint(255, 0.376, 0.188, 0.0)

```

```

opacityTransferFunction = vtkPiecewiseFunction()
opacityTransferFunction.AddPoint(0, 0.0)
opacityTransferFunction.AddPoint(255, 1.0)

# New TF (Clean)
colorTransferFunction2 = vtkColorTransferFunction()
colorTransferFunction2.AddRGBPoint(0, 0.0, 0.667, 0.0)
colorTransferFunction2.AddRGBPoint(96, 0.925, 0.463, 0.0)
colorTransferFunction2.AddRGBPoint(130, 0.667, 0.463, 0.0)
colorTransferFunction2.AddRGBPoint(255, 0.376, 0.188, 0.0)

opacityTransferFunction2 = vtkPiecewiseFunction()
opacityTransferFunction2.AddPoint(0, 0.0)
opacityTransferFunction2.AddPoint(32, 0.0)
opacityTransferFunction2.AddPoint(70, 0.3)
opacityTransferFunction2.AddPoint(255, 1.0)

TFList=[]
TFList.append((colorTransferFunction,opacityTransferFunction))
TFList.append((colorTransferFunction2,opacityTransferFunction2))

volumeProperty = vtkVolumeProperty()
volumeProperty.SetColor(colorTransferFunction)
volumeProperty.SetScalarOpacity(opacityTransferFunction)
volumeProperty.ShadeOn()
volumeProperty.SetInterpolationTypeToLinear()

volumeMapper = vtkFixedPointVolumeRayCastMapper()
volumeMapper.SetInputConnection(reader.GetOutputPort())

# Set to 1 for Part D testing
volumeMapper.SetAutoAdjustSampleDistances(1)

volume = vtkVolume()
volume.SetMapper(volumeMapper)
volume.SetProperty(volumeProperty)

ren1.AddVolume(volume)
ren1.SetBackground(colors.GetColor3d('Wheat'))
ren1.GetActiveCamera().Azimuth(45)
ren1.GetActiveCamera().Elevation(30)
ren1.ResetCameraClippingRange()
ren1.ResetCamera()

renWin.SetSize(600, 600)
renWin.SetWindowName('SimpleRayCast')
renWin.Render()

print("Press 't' to toggle transfer function.")
print("Press 'i' to toggle interpolation.")
iren = vtkRenderWindowInteractor()
iren.SetRenderWindow(renWin)
iren.SetInteractorStyle(MyInteractorStyle(renWin,volumeProperty,TFList))
iren.SetDesiredUpdateRate(50)
iren.Start()

if __name__ == '__main__':
    main()

```